CoperniCoaches

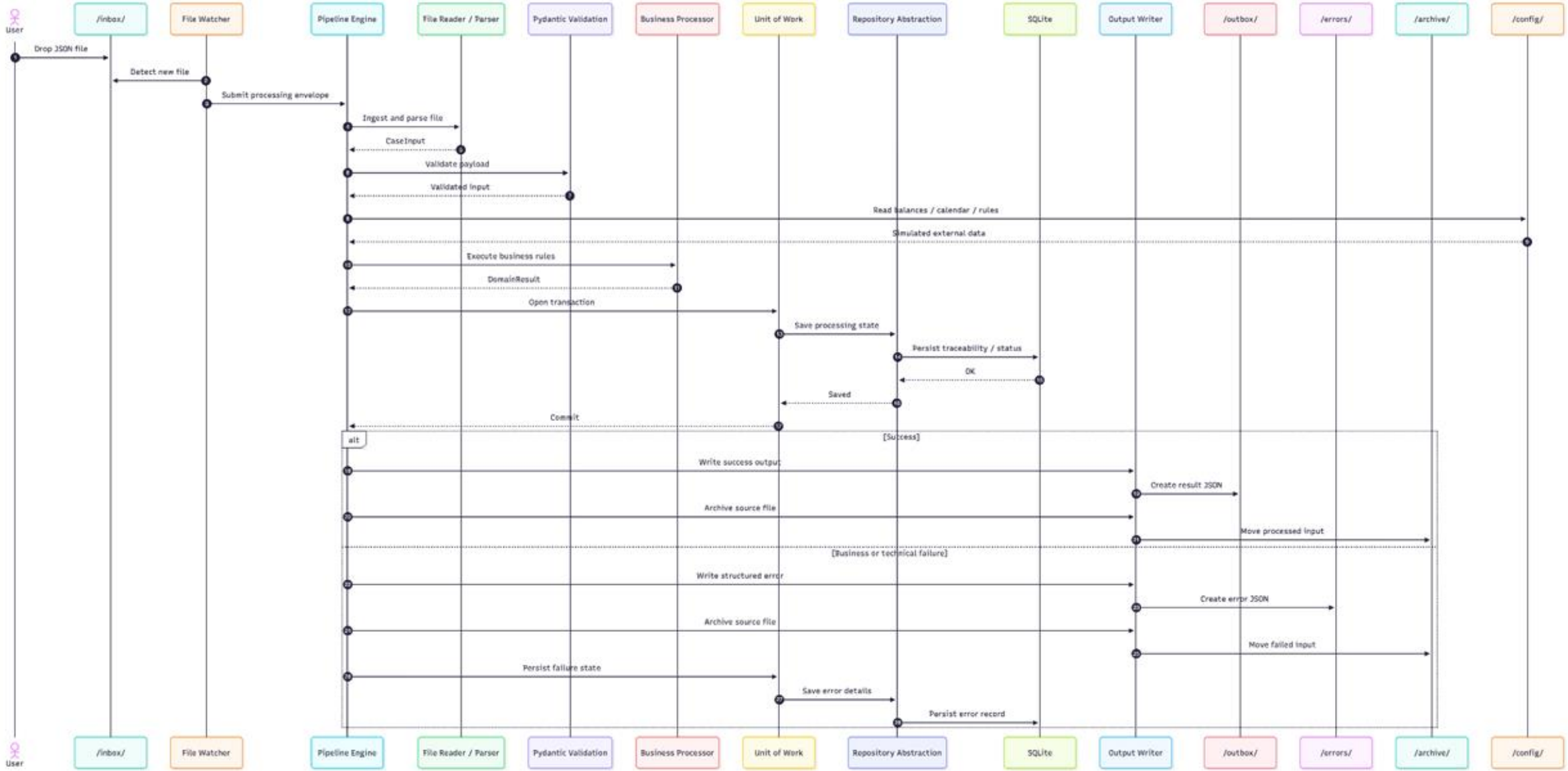
Dia 6 - Lunes 23 de marzo

Capgemini

CoperniCoaches - Dia 6

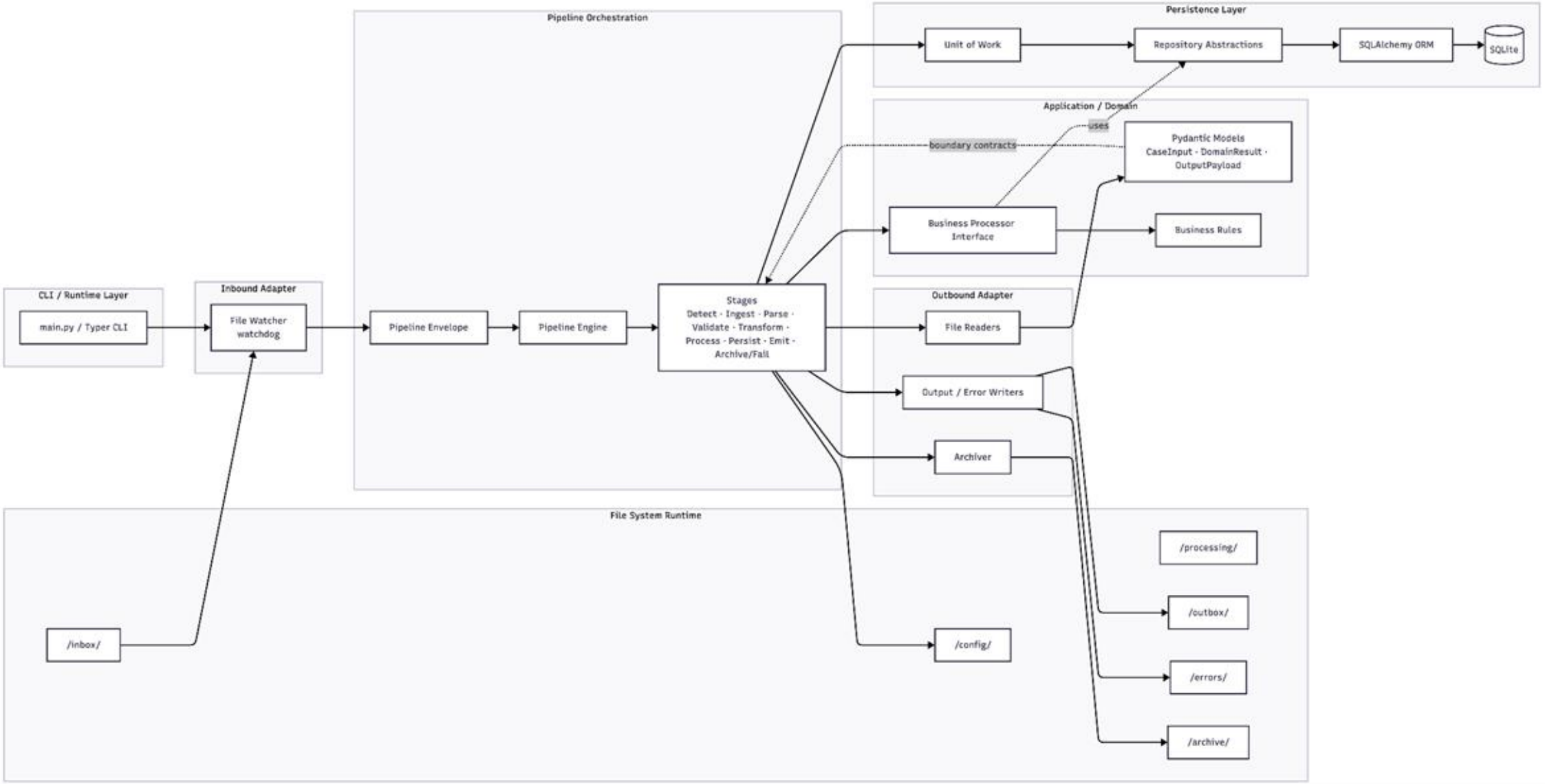
Lunes 23 de marzo 2026

CoperniCoaches - Dia 6



Lunes 23 de marzo 2026

CoperniCoaches - Dia 6



Gestor de archivos — Miller Columns

Descripción y capacidades

Descripción

Gestor de archivos en consola, local, con navegación por Miller Columns (tres columnas):

- izquierda → directorio padre
- central → directorio actual
- derecha → contenido (hijos)

Visual (colores), reactivo y navegable por teclado.

Capacidades mínimas

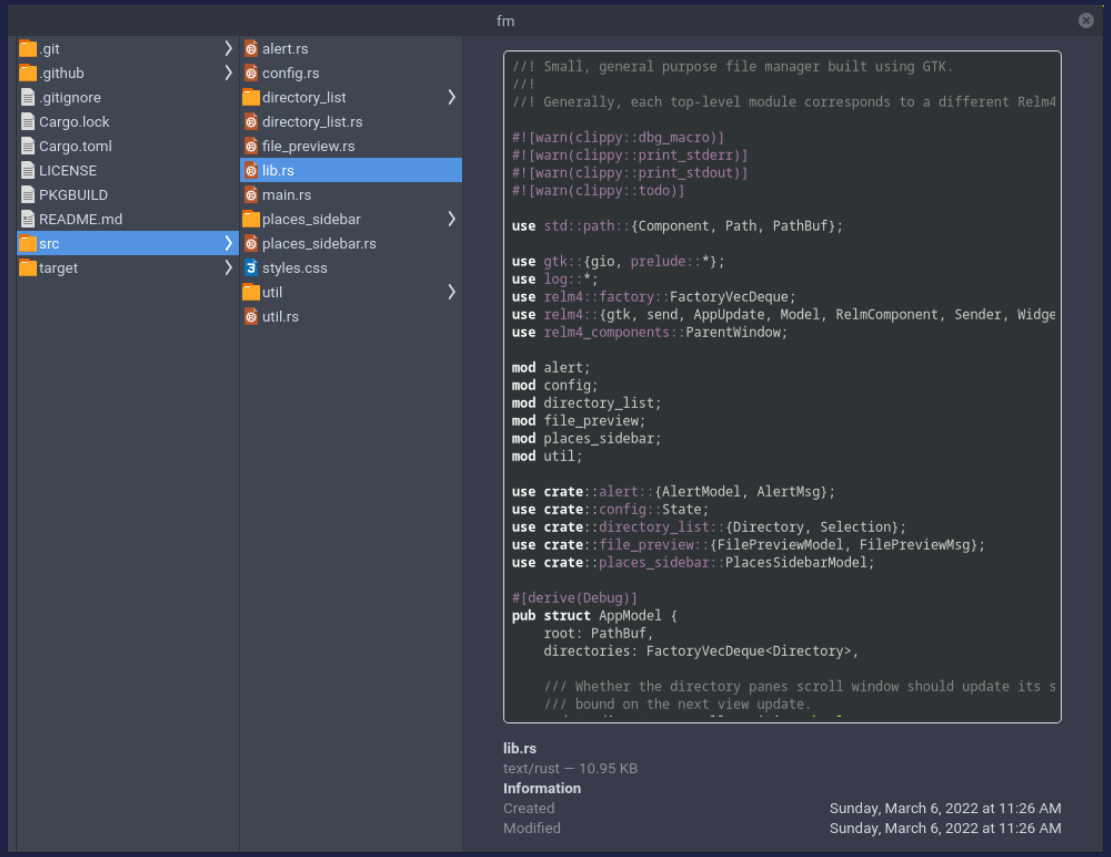
- navegación arriba / abajo / entrar / salir
- visualización clara de rutas
- diferenciación visual (colores, tipos de archivo)
- lectura del sistema de archivos local
- actualización dinámica al navegar

Modelo conceptual

- sistema de ficheros jerárquico
- navegación por Miller Columns (column view)
- separación entre:
  - lectura del sistema
  - estado de navegación
  - renderizado en consola

Referencia

Miller Columns (column view): https://en.wikipedia.org/wiki/Miller_columns



Gestor de archivos — Atrium

Implementacion en Python

```
Path: /home/iwk/src/copernicoaches

Parent                          [* Current *]                   Preview
-------------------------------------------------------------------------------
  __scratchpad__                  .git                            .venv
  __workbench__                   .github                         atrium
  AIDev-demo                      .venv                           images
  aihub                           .vscode                         specs
  backup_tee.git                  ac_checkaccountbalance          src
  bender                        ▶ atrium                          target
  bob                             dia1                            tests
  bobby                           dia2                            .gitignore
  boely                           dia3                            .python-version
  brainflip                       dia4                            Cargo.lock
  brainflip_old                   dia5                            Cargo.toml
  brownie                         dia6                            pyproject.toml
  buff                            jano                            README.md
  clipstrip                       madriguera                      uv.lock
  codescribe                      .gitignore
  complex                         README.md
  conjunctapose
▶ copernicoaches
  cyphersprite
  dataspec
  delta-fiber
  devloop
  dict-ai-te
  dotnetdemo
  egui
```

Gestor de archivos — Atrium

Implementacion en Rust

Parent	[* Current *]	Preview
📁 __scratchpad__	📁 .git	📁 .venv
📁 __workbench__	📁 .github	📁 atrium
📁 AIDev-demo	📁 .venv	📁 images
📁 aihub	📁 .vscode	📁 specs
📁 backup_tee.git	📁 ac_checkaccountbalance	📁 src
📁 bender	▶ 📁 atrium	📁 target
📁 bob	📁 dia1	📁 tests
📁 bobby	📁 dia2	📄 .gitignore
📁 boely	📁 dia3	📄 .python-version
📁 brainflip	📁 dia4	📄 Cargo.lock
📁 brainflip_old	📁 dia5	📄 Cargo.toml
📁 brownie	📁 dia6	📄 pyproject.toml
📁 buff	📁 jano	📄 README.md
📁 clipstrip	📁 madriguera	📄 uv.lock
📁 codescribe	📄 .gitignore	
📁 complex	📄 README.md	
📁 conjunctapose		
▶ 📁 copernicoaches		
📁 cyphersprite		
📁 dataspec		
📁 delta-fiber		
📁 devloop		
📁 dict-ai-te		
📁 dotnetdemo		
📁 egui		

Ingeniería de contexto en desarrollo con IA

Principio clave
La calidad del sistema depende de cómo se estructura el contexto, no solo del modelo.

Enfoque efectivo
Generar especificaciones intermedias, autocontenidas, antes de implementar.

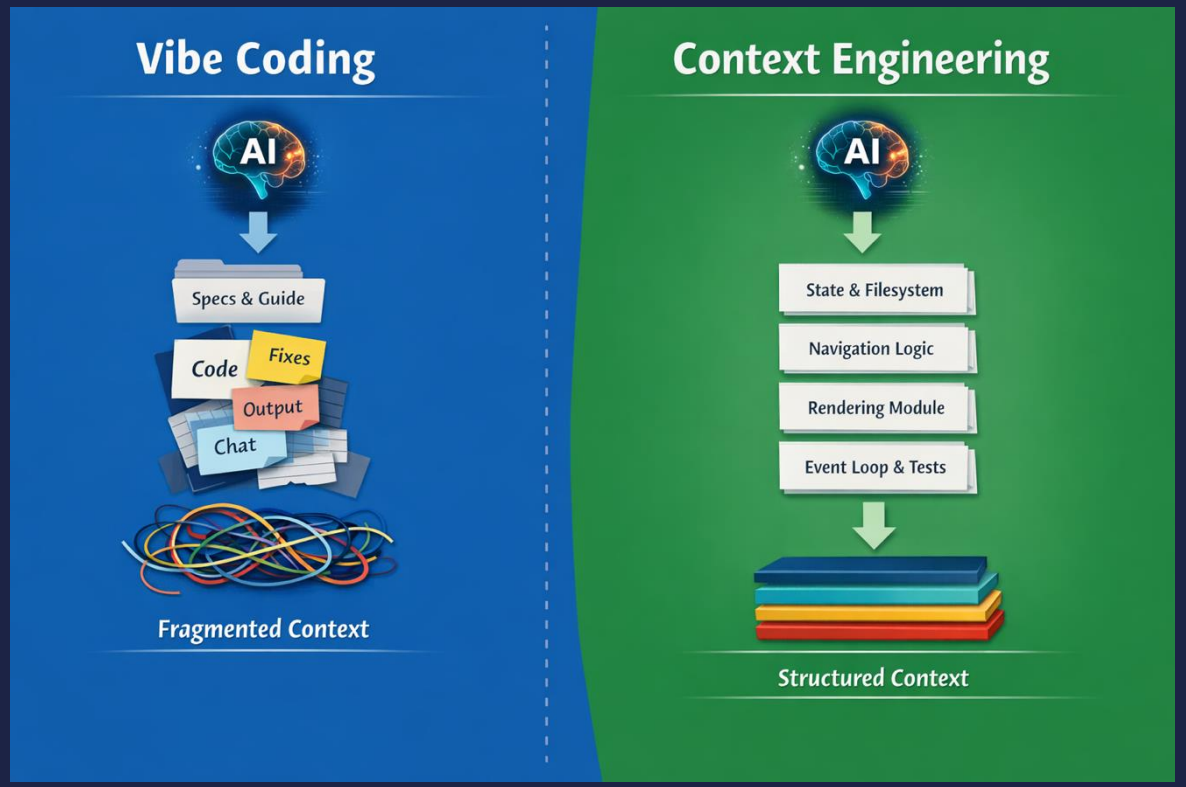
Qué se consigue
- Cada paso trabaja con contexto completo y relevante
- La arquitectura se preserva (estado, navegación, renderizado separados)
- Se evitan desviaciones acumulativas
- El desarrollo se vuelve verificable en cada fase

Aplicación (Atrium)
Dividir el sistema en:
estado + filesystem → navegación → renderizado → composición

Resultado
De iteración reactiva a construcción controlada y reproducible

Una exposición clara sobre la diferencia entre el typecoding basado en prompting ad hoc y la ingeniería de contexto estructurada puede encontrarse en el artículo:

Dia6/Ingeniería de contexto en generación de código.md



Ejecución individual y método

Objetivo y entrega

Objetivo

Demostrar que el participante no solo implementa, sino que trabaja con método, control y criterio.

Entrega obligatoria (plantilla)

El participante debe acompañar la solución con:

- delimitación del problema
- contexto preparado
- descomposición en pasos
- especificación mínima
- interacción con IA
- verificación
- gestión del fallo
- artefactos generados
- explicación breve reutilizable

Qué se evalúa

- capacidad de acotar el problema
- calidad del contexto (filtrado, estructurado)
- trabajo por pasos
- definición previa antes de implementar
- verificación real de resultados
- capacidad de corregir enfoque
- trazabilidad del proceso

Criterio central

Uso correcto de IA = reducción de entropía

El trabajo debe volverse:

- más claro
- más acotado
- más verificable
- más reutilizable

Resultado esperado

Tres perfiles emergen:

- **no alcanza nivel** → ejecución sin control
- **ejecutor competente** → resuelve correctamente
- **perfil de coach** → estructura, verifica y deja método reusable

Para la especificacion functional ver documento: **Dia6/Especificación funcional - Gestor de archivos en consola con Miller Columns.md**

Para la especiciacion de Guia de arquitectura ver documento**: Dia6/Guía de arquitectura - Aplicación de consola.md**

La plantilla del informe que se presenta en paralello con la solucion de ejercicio se encuentra en

Dia6/Informe individual de uso de IA.md

¡Gracias!

Company Confidential © Capgemini 2026. All rights reserved